Compilation Project

ABBASSI, BOUZID

March 2026

Partie 7 – A simple stack language

Exercise 1 (expl)

A stack is a data structure following the *LIFO* principle (*Last In, First Out*): the last element pushed onto the stack is the first one to be removed.

The usual operations on a stack are:

- **push**: add an element on the top of the stack.
- **pop**: remove the top element of the stack.
- **top** (or **peek**): read the top element without removing it.
- **is_empty**: test whether the stack is empty.

Exercise 2 (expl)

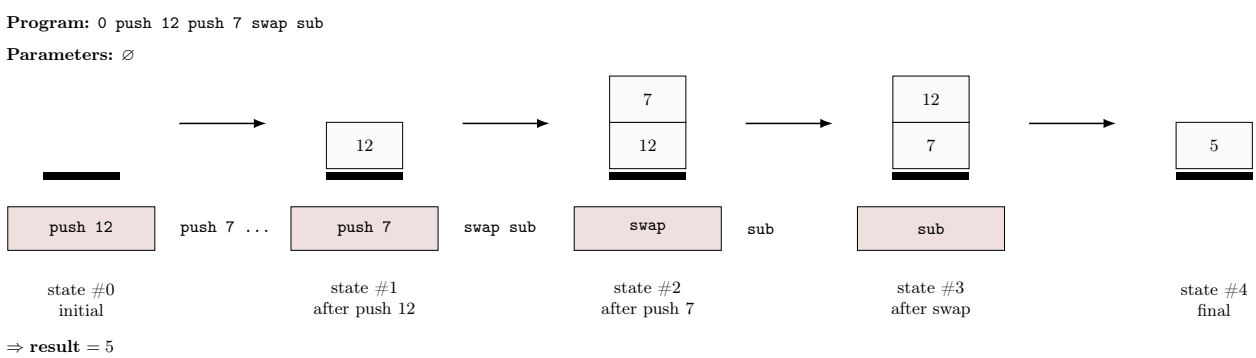

Figure 1: Detailed execution of `0 push 12 push 7 swap sub`.

Exercise 3

Question 3.1 (expl)

The semantics of programs is defined by three rules.

1. If the program expects i arguments but receives n arguments with $i \neq n$, then execution immediately fails and returns `Err`.
2. If the number of arguments is correct, then the arguments are first placed on the initial stack, and the instruction sequence Q is executed. If this execution leads to a runtime error, then the whole program returns `Err`.

3. If the number of arguments is correct and the execution terminates normally on a non-empty stack of the form $v :: S$, then the result of the program is the top value v.

   In short:

- wrong number of arguments $\Rightarrow$ `Err`,
- runtime error during execution $\Rightarrow$ `Err`,
- successful execution with a non-empty final stack $\Rightarrow$ result is the top of the stack.

Question 3.2 (math)

A case is missing: when the number of arguments is correct, execution terminates normally, but the final stack is empty. In that case, there is no result to return, so the program must return `Err`.

The missing rule is:

$$\frac{Q,\ v_1 :: \cdots :: v_n :: \varnothing \to^* \varnothing,\ \varnothing}{v_1, \ldots, v_n \vdash n, Q \Rightarrow \mathrm{Err}}$$

Question 3.3 (math)

We now define the small-step semantics of instruction sequences.

Push

$$\frac{}{\texttt{push}\ k.Q,\ S \to Q,\ k :: S}$$

Pop

$$\frac{}{\texttt{pop}.Q,\ k :: S \to Q,\ S} \qquad \frac{}{\texttt{pop}.Q,\ \varnothing \to \mathrm{Err}}$$

Swap

$$\frac{}{\texttt{swap}.Q,\ k_1 :: k_2 :: S \to Q,\ k_2 :: k_1 :: S}$$

$$\frac{}{\texttt{swap}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{swap}.Q,\ k :: \varnothing \to \mathrm{Err}}$$

Add

$$\frac{}{\texttt{add}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 + k_2) :: S}$$

$$\frac{}{\texttt{add}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{add}.Q,\ k :: \varnothing \to \mathrm{Err}}$$

Sub

$$\frac{}{\texttt{sub}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 - k_2) :: S}$$

$$\frac{}{\texttt{sub}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{sub}.Q,\ k :: \varnothing \to \mathrm{Err}}$$

Mul

$$\frac{}{\texttt{mul}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \times k_2) :: S}$$

$$\frac{}{\texttt{mul}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{mul}.Q,\ k :: \varnothing \to \mathrm{Err}}$$

Div

$$\frac{k_2 \neq 0}{\texttt{div}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \div k_2) :: S}$$

$$\frac{}{\texttt{div}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{div}.Q,\ k :: \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{div}.Q,\ k_1 :: 0 :: S \to \mathrm{Err}}$$

Rem

$$\frac{k_2 \neq 0}{\texttt{rem}.Q,\ k_1 :: k_2 :: S \to Q,\ (k_1 \bmod k_2) :: S}$$

$$\frac{}{\texttt{rem}.Q,\ \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{rem}.Q,\ k :: \varnothing \to \mathrm{Err}} \qquad \frac{}{\texttt{rem}.Q,\ k_1 :: 0 :: S \to \mathrm{Err}}$$

Final configuration When the instruction sequence is empty, execution is finished:

$$\varnothing,\ S$$

is a final configuration.

Exercise 5

Question 5.1

At this stage, variables cannot be compiled because the current version of *Pfx* does not provide any instruction to access an environment. Therefore, the compilation schema is defined only for variable-free arithmetic expressions.

Instruction sequences are built with the constructor "." and the empty sequence is written $\emptyset$. We define the compilation as:

$$\mathcal{C}(e) = (0, \mathcal{G}(e, \emptyset))$$

where $\mathcal{G}(e, Q)$ means "generate the code that evaluates e and then continues with the instruction sequence Q".

We use the following correspondence between `Expr` binary operators and `Pfx` instructions:

$$\widehat{+} = \texttt{add}, \qquad \widehat{-} = \texttt{sub}, \qquad \widehat{*} = \texttt{mul}, \qquad \widehat{/} = \texttt{div}, \qquad \widehat{\%} = \texttt{rem}.$$

The translation rules are:

$$\mathcal{G}(n, Q) = \texttt{push}\ n.Q$$

$$\mathcal{G}(-e, Q) = \mathcal{G}(e, \texttt{push}\ 0.\texttt{sub}.Q)$$

$$\mathcal{G}(e_1 \oplus e_2, Q) = \mathcal{G}(e_2, \mathcal{G}(e_1, \widehat{\oplus}.Q)) \qquad (\oplus \in \{+, -, *, /, \%\})$$

Variables are not compiled at this stage: $\mathcal{G}(x, Q)$ is undefined because the current *Pfx* machine has no instruction to fetch the value of a variable.

The order in the binary-operation rule is important. In *Pfx*, arithmetic instructions use the top element of the stack as their first argument and the second element as their second argument. Therefore, to compile $e_1 - e_2$, we must first push the value of e_2 and then the value of e_1, so that `sub` computes $e_1 - e_2$.

The intended correctness property is the following: if e is a variable-free expression whose value is v, then executing $\mathcal{G}(e, Q)$ on a stack S has the same effect as executing Q on the stack $v :: S$. In particular, executing $\mathcal{C}(e)$ leaves the value of e on top of the initial empty stack.